

Designing a Community Security Alert System for a University Campus: Analysis, Architecture, and Lessons Learned

Felipe Jose Garzon Herrera, 20251020132, Juan Esteban Quintero Gordillo, 20251020137,
Henry Samuel Garrido Medina, 20251020125, Gabriel Mateo Cusba Marin, 20251020128

Systems Analysis & Design, Semester 2026-I

Eng. Carlos Andrés Sierra, M.Sc.

Computer Engineering Program, Universidad Distrital Francisco José de Caldas, Bogotá D.C., Colombia

Abstract—University campuses in Latin America face persistent safety challenges rooted in the absence of real-time, community-driven incident reporting tools. At Universidad Distrital Francisco José de Caldas, systematic data collection revealed three compounding problems: a strong under-reporting culture driven by high-friction reporting channels, a concentration of incidents in specific zones and hours, and a lack of centralized incident management that produces average security response delays of 11.4 minutes. This paper presents the Community Security Alert System (CSAS), a crowd-sourced mobile platform designed to address these gaps through a microservices-based architecture with a hybrid AI-and-human verification pipeline and geofenced alert dispatch. The proposed design projects a reduction in response time to under 5 minutes and an increase in incident recording completeness from 41% to above 85%, establishing a replicable methodology for campus security systems in similar institutional contexts.

Index Terms—campus security, crowd-sourced incident reporting, microservices, real-time alerts, systems analysis, systems design.

1. INTRODUCTION

Campus security is a problem that most universities acknowledge but few solve effectively. At Universidad Distrital Francisco José de Caldas, a public university in Bogotá D.C., the security team relies on phone calls and walk-up reports to respond to incidents — a process that takes an average of 11.4 minutes from the moment an event occurs to the moment the response team is notified. During that time, a situation can escalate significantly.

Several platforms have attempted to address similar problems in the Bogotá metropolitan area, and their limitations are instructive. The Policía Nacional’s Vigila App integrates directly with law enforcement but requires users to place a phone call, making it too slow for fast-moving campus situations [1]. Neighborhood WhatsApp groups achieve high adoption but operate without any verification mechanism, routinely generating false alarms and panic [2]. SIURE UD, the university’s own internal system, is familiar to staff but operates exclusively through email and phone, with no mobile interface or real-time capability [1]. The pattern across these platforms is consistent: each solves part of the problem but creates a different one in its place.

The theoretical framing for this project comes from two foundational works. Checkland’s soft systems methodology [2] helped us treat campus security not as a purely technical problem but as a sociotechnical system in which human behavior, institutional culture, and technology interact in complex ways — a perspective that proved essential during the analysis phase. Meadows’ work on feedback dynamics [3] provided the vocabulary to understand how a crowd-sourced alert system can either amplify its own usefulness through reinforcing loops, or undermine it through alert fatigue if poorly calibrated. On the technical side, the systems engineering life-

cycle framework of ISO/IEC/IEEE 15288:2015 [4] structured our progression from analysis to design, while Bass, Clements, and Kazman [7] and Dragoni et al. [8] informed our architectural decisions, particularly the choice of a microservices-based decomposition over a monolithic design. The mixed-methods data collection strategy was guided by Creswell and Creswell [5], whose framework for triangulating qualitative and quantitative evidence helped us assess the validity of findings derived from a relatively small sample.

This paper presents the Community Security Alert System (CSAS), a crowd-sourced mobile and web platform that addresses the identified gaps. The paper covers two phases of the project: a systems analysis phase in which we characterized the problem empirically, and a systems design phase in which we translated those findings into a technical architecture. Our central argument is that the design decisions made in each phase are only meaningful because they are grounded in real data collected from the campus community — not assumptions.

2. METHODS AND MATERIALS

A. Systems Analysis

To characterize the campus security problem, we designed a data collection plan that combined six complementary methods simultaneously. The goal was to avoid relying on any single perspective, since each method captures a different dimension of the problem. Structured interviews with five security staff members and two administrative staff members revealed the operational reality of campus security from the inside: communication delays, lack of centralized records, and the need for precise incident location data. A student survey ($n = 20$) measured safety perception, incident experience, and willingness to use a reporting application, and showed that

35% of respondents had experienced or witnessed an incident, with all reported events occurring during afternoon or night hours. Direct observation across four campus zones — main entrance, library area, parking lot, and dormitory — over eight sessions confirmed that the parking lot at night is the highest-risk zone, with two suspicious events observed both coinciding with poor lighting and low foot traffic. Document analysis of institutional security records confirmed a significant under-reporting gap: official records show far fewer incidents than the survey and observation data suggest. Benchmarking of three existing platforms in Bogotá extracted transferable design insights while identifying the gaps that none of them fully close. Finally, weather monitoring over one week suggested a preliminary correlation between rainy nights and incident occurrence, motivating a weather-aware component in the design.

The convergence of three independent methods — interviews, survey, and observation — on the same three problems gave us confidence in the findings: under-reporting due to high friction, night-time and spatial concentration of incidents, and the absence of a centralized real-time tool. When independent methods agree, the conclusion is stronger than any single source of evidence can provide.

Figure 1 shows the stakeholder context diagram, mapping the information flows between the six groups that interact with the CSAS: students, professors and administrative staff, the campus security team, local authorities, system administrators, and university management.

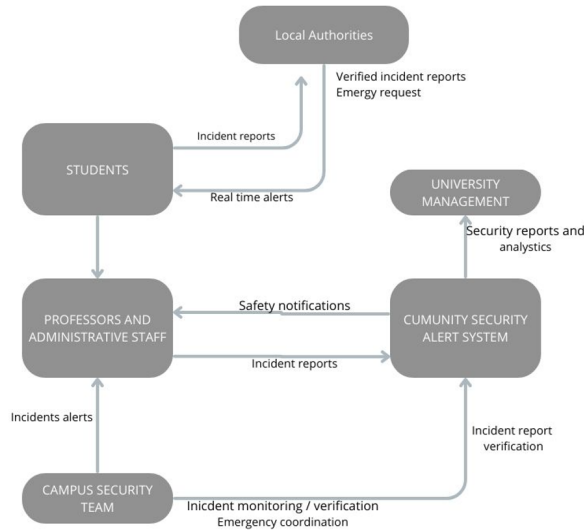


Figure 1. Stakeholder relationships and information flows in the CSAS. Students are the primary reporters; security staff verify and coordinate; management consumes analytics.

B. System Design

The design phase translated the three empirical findings into three corresponding architectural decisions. The under-reporting culture is addressed by a mobile-first interface that requires no more than three interactions to submit a report

(NFR-06), and by an anonymous reporting option for sensitive incident types. The spatial and temporal concentration of incidents is addressed by a geofenced alert dispatcher that applies zone-weighted and time-weighted priority logic, automatically elevating the urgency of night-hour alerts from the parking lot and dormitory zones. The absence of a centralized tool is addressed by a microservices-based, event-driven architecture that integrates with the existing SIURE UD institutional system rather than replacing it.

The architecture is organized across five layers. The presentation layer encompasses the mobile application (React Native) and the administrative web portal. The application layer is an API gateway (Node.js / Nginx) that handles authentication, rate limiting, and routing. The service layer contains the five core microservices — Incident, Verification, Dispatcher, User, and Analytics — which communicate asynchronously through a RabbitMQ message broker to ensure loose coupling between report ingestion and alert dispatch. The data layer uses PostgreSQL with the PostGIS spatial extension, which supports the geospatial queries required for zone-weighted routing and security heatmap generation. The integration layer exposes authenticated endpoints to SIURE UD, local authorities, and a Twilio SMS gateway that provides a fallback delivery channel for users without smartphone access.

The verification pipeline deserves particular attention because it sits at the center of the most critical design trade-off in the system. If verification is too strict, legitimate incidents get delayed. If it is too loose, false alerts flood the system and users start ignoring everything — a phenomenon known as alert fatigue [3]. Our solution is a hybrid AI-and-human pipeline: an AI plausibility scorer performs initial metadata validation and assigns a confidence score, while a human moderator reviews low-confidence reports and can override the scorer at any stage. High-severity incident types (assault, medical emergency) have a priority bypass path that skips AI scoring entirely and goes directly to the security team. The NFR-08 requirement formalizes the quality bar: no alert is broadcast as verified without at least two independent community confirmations or one administrative approval.

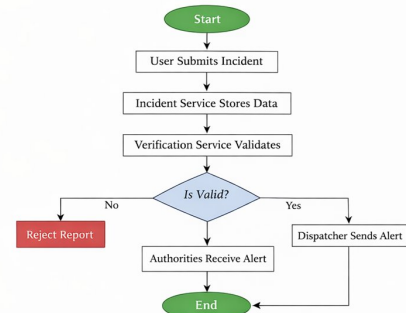


Figure 2. Main process flow. A submitted incident passes through the Verification Service; valid incidents trigger geofenced dispatch while rejected reports are logged for auditing.

Figure 2 illustrates this flow. Figure 3 shows the full layered architecture.

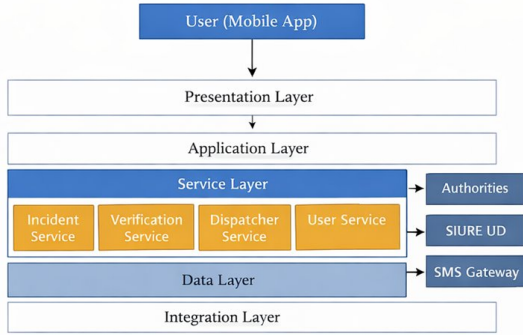


Figure 3. CSAS layered architecture. The Service Layer hosts the five core microservices; external integrations connect through the Integration Layer.

One important limitation of our analysis is the survey sample size ($n = 20$), which is too small for statistical generalization. We also did not audit mobile connectivity coverage across campus zones, which is a prerequisite risk for the notification delivery architecture. These are acknowledged open points that the implementation phase must address before deployment.

3. RESULTS AND DISCUSSION

A. Gap Analysis

The clearest result of the analysis phase is a set of measurable operational gaps between the current state of campus security and the targets the CSAS is designed to achieve. Table 1 presents these gaps quantitatively. The response time gap of -6.4 minutes is the most critical: reducing from 11.4 minutes to under 5 minutes would fundamentally change the security team’s ability to intervene during fast-developing incidents.

Table 1. Operational Gap Analysis: Current State vs. CSAS Design Targets

Dimension	Baseline	Target	Gap
Response time	11.4 min	<5 min	-6.4 min
Reporting rate	41%	>85%	+44 pp
App adoption	~12%	$\geq 60\%$	+48 pp
Delivery success	~87%	>99.9%	+12.9 pp

B. Comparison with Existing Platforms

Table 2 compares the CSAS against the three existing platforms reviewed during benchmarking. The CSAS is the only platform in this comparison that combines all four capabilities: mobile-first interface, real-time geofenced dispatch, incident verification, and institutional integration.

Table 2. Feature Comparison: CSAS vs. Existing Platforms in Bogotá

Platform	Mobile	Real-time	Verified	Institutional
Vigila App	Partial	No	No	Yes
WhatsApp nets	Yes	Yes	No	No
SIURE UD	No	No	Yes	Yes
CSAS	Yes	Yes	Yes	Yes

C. Key Performance Indicators

Since no implementation exists at this stage, the KPIs in Table 3 represent design targets rather than measured outcomes. They are included here because they formalize the success criteria against which the future system will be objectively evaluated, and because comparing the baselines with the targets already constitutes a result: it quantifies exactly how far the current state is from what is needed, which is useful information in itself.

Table 3. Key Performance Indicators and Measurement Plan

KPI	Target	Method	Freq.
Alert latency	<30 s (NFR-01)	Event logs	Daily
Delivery rate	>99.9% (NFR-02)	Push receipts	Daily
Availability	99.9% (NFR-03)	Monitoring	Cont.
Reporting rate	>85% logged	DB records	Monthly
Adoption rate	>60% students	MAU analytics	Monthly
False alarm rate	$\leq 5\%$	Staff review	Weekly

D. Discussion

The most important insight from this project is that the technical architecture is only as strong as the empirical evidence behind it. Every major design decision — the three-interaction submission flow, the night-weighted dispatcher, the RabbitMQ message broker, the hybrid verification pipeline — was made in direct response to a specific finding from the analysis phase. If we had started with the design, we would likely have built something technically correct but practically useless.

The feedback dynamics identified in the analysis also shape how we think about deployment. The CSAS depends on a reinforcing loop to become effective: as more users report incidents, alerts become more useful, which encourages more reporting. But this loop only activates above a critical adoption threshold of approximately 20–25% of the campus population. Below that threshold, the system depends almost entirely on security staff to feed the verification pipeline. This means that community engagement is not a nice-to-have feature of the CSAS — it is a structural prerequisite for the system to work as designed, and it cannot be achieved by the technical team alone.

Three assumptions in the design remain unvalidated and represent the primary risks for the implementation phase. First,

we assumed that users will report incidents if the process is easy enough; our survey supports this directionally (100% of incident-experienced participants said they would use the app), but real-world behavior under operational conditions may differ. Second, we assumed the verification pipeline can operate within a 45-second latency budget; this depends on AI model performance and infrastructure configuration that have not been tested. Third, we assumed reliable mobile connectivity across all campus zones; we did not audit signal coverage, and dead zones could break the notification delivery guarantees.

4. CONCLUSIONS

This paper presented the analysis and design of the Community Security Alert System, a crowd-sourced platform for campus safety at Universidad Distrital Francisco José de Caldas. The work demonstrates that combining six independent data collection methods — even with a small sample — can produce a robust and consistent characterization of a sociotechnical problem. When interviews, surveys, and direct observation converge on the same three problems (under-reporting, night-time concentration, and absence of a centralized tool), the evidence is strong enough to make design decisions with confidence. The proposed microservices architecture addresses each of these problems directly and is grounded in the empirical evidence at every decision point.

The most important lesson from this project is that systems analysis is not a formality before the real work begins — it is the foundation on which the design depends. The KPI targets, the verification pipeline calibration, the adoption threshold, and the night-weighted dispatcher all came from data, not intuition. For future work, we recommend expanding the student survey to at least 200 participants, conducting a campus WiFi coverage audit before finalizing the notification delivery architecture, and establishing a formal data processing agreement with SIURE UD and university legal counsel as prerequisites to deployment — because the 60% adoption target cannot be achieved without institutional endorsement.

4. REFERENCES

- [1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.
- [2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.
- [3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.
- [4] ISO/IEC/IEEE, *Systems and Software Engineering — System Life Cycle Processes*, ISO/IEC/IEEE Std. 15288:2015, 2015.
- [5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.
- [6] Congreso de Colombia, *Ley 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.
- [7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.
- [8] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” *Present and Ulterior Software Engineering*, pp. 195–216, 2017.